

## Session control (Main module)

- first to run and calls others as needed, requests user input through Display/Input parser
- modes: static calculation request, static/optimal plan building, output
- permanent: presents current plan as ASCII above user choices via Imagery maker and Display
- error details are logged via decorator Debugger

## Display and Input parser layer

- reads config files (if debugging) THEN interactive input
- Validates/Sanitize input
- maintains a Read evaluate print loop and a display screen for the current state of the plan

## Debugger decorator

- handles exceptions and inconsistencies
- creates report for the output part of imagery maker

## Imagery maker

- builds a technical drawing from the plan in ASCII art

## Output

- prepares exporting to PDF, CSV etc.

## Plan collection

- holds all constructed components in memory

## The component factory

- Fetches req.s for each obj and requests it from Session control when it asks for a new obj.
- obtains constants from variables file
- creates the objects and IF Optimat=ON, serves them to the Simulation, if REDO, then looks for what variables it's allowed to change, gives Session control "user data will fail, but can succeed if X is Y" or "print Impossible", IF Optimal=OFF, it links Ses Contr with relevant function calls

## Simulation

- STATIC Tests obj for compatibility and disaster events by calling the possible triggers and calculating results across plan, then marks pass/fail for proceed/redo logic

## Variables file

**Conduits**  
- material, length, diameter, parallel/series, heat rating (based on Amps & ambient), connections, env, safeties

**Outlets**  
- V & A ratings, parallel/series, neighbors, safeties, env

**Grounding**  
- space, soil, humidity, rod and strip details

**Lights**  
- Power draw, parallel/series, neighbors, env, safeties

**Fuses**  
- expected V & Amp on circuit  
- limits for short-circuit and lightning  
- env details

**Electrical control objects**  
- desired min/max V and Amps  
- relevant circuit elements for monitoring/control  
- safeties and integration methods  
- other details

Solar Panels For a lot later!  
NOT EVEN A THOUGHT HERE

```
Component {  
    type: "grounding_rod" | "conduit" | "outlet" | "breaker"  
    | ...  
    properties: { ...type-specific values... }  
    connections: [previous obj in chain, input_conduit,  
output_conduit, next obj in chain or None]  
    constraints: [ ...validation and safety rules... ]  
    Functions_it_can_call: [...calculations and triggers...]  
}
```

PER OBJECT:  
Env: average temp summer / winter, humidity, light, wind;  
Calculations: ohm law, other maths...  
Triggers example: if rating X, then cause Y, affects Z circuit (conduit melt, shortcircuits, lightning, etc)  
Source of triggers: grid instability, user adds consumer with incorrect ratings/malfunction, environmental by season

# Electrical Calculator